

수식	의미	챕터
$Physical = Base + Offset$ (if $Offset < Bound$)	Base/Bound MMU 주소 변환	Ch1
Mode bit: 0=kernel, 1=user (PSW)	Dual mode operation	Ch2
인자: $ebp+8, ebp+12, \dots$	x86 cdecl 인자 위치 (caller frame)	Ch3
지역변수: $ebp-4, ebp-8, \dots$	x86 cdecl 지역 변수 위치	Ch3
반환값 = <code>%eax</code>	x86 cdecl 반환값 위치	Ch3
$\tau_{\{n+1\}} = \alpha \cdot \tau_n + (1-\alpha) \cdot \tau_n$	Exponential moving average (SJF burst 예측)	Ch5
$TS_i = (W_i / S_\Phi) \times P$	WRR/CFS time slice	Ch5
$VR(\tau_i, t) = (W_0/W_i) \times PR(\tau_i, t)$	CFS virtual runtime	Ch5
nice 1단계 = CPU $\pm 10\%$	Linux CFS nice rule	Ch5
$lag = \text{실제_받은_CPU} - \text{이상적_GPS_CPU}$	Fair share fairness 측정	Ch5
$C_\tau(t1, t2) = (W_\tau/S_\Phi) \times (t2-t1)$	GPS perfect fairness	Ch5
Need = Max - Allocation	Banker's algorithm	Ch7
Block size = Payload + Header + Padding (+ Footer)	malloc block 구조	Ch9
4가지 coalescing case: (Alloc/Free) $\times$ (Alloc/Free)	Boundary tag coalescing	Ch9
$Physical = PFN \parallel Page_Offset$	Paging address translation	Ch10
$Physical = Base_seg + (Page_Table[PFN] \parallel Offset)$	Paged segmentation	Ch10
$t_{em} = \alpha \cdot (t_c + t_m) + (1-\alpha) \cdot (2t_c + 2t_m)$	TLB Effective Access Time	Ch10
$t_{em} = t_c + t_m + (1-\alpha) \cdot t_m$ (전개식)	TLB EAT 전개	Ch10
$t_{eff_acc} = p \cdot t_{mem} + (1-p) \cdot t_{fault}$	Demand paging EAT	Ch11
Working set = last τ 초 동안 참조된 모든 page	Working set 정의	Ch11
32-bit, 1KB page, 4B PTE $\rightarrow$ PT 크기 16MB	Paging table 크기 폭증 예시	Ch10
64-bit, 4KB page $\rightarrow$ 64K TB	64-bit page table 비현실성	Ch11

Disk: seek 10~20ms + rotation 0~16ms + transfer 8~40μs	Disk 시간 (2012 기준)	Ch12
Flash: read 20μs / write 200μs / erase 2000μs	Flash 동작 시간	Ch12
Path parsing: inode 2 (root) → directory 내 용 → inumber → ...	Unix path → file 변환	Ch13
4.3 BSD inode: 12 direct + 1 indirect + 1 double-indirect	Multi-level indexed file	Ch14
4KB block, 4B index → direct 48KB, indirect 4MB, double 4GB	4.3 BSD 파일 크기 한 계	Ch14
DRAM vs Disk = 100,000배 (지구↔달)	File system 디자인의 동기	Ch14
FFS bandwidth: 14~47% (vs S5FS 2~5%)	Disk awareness의 효 과	Ch14
LFS write bandwidth: 70%	Log-structured write 효율	Ch14
2TB drive ↔ 64MB bitmap	Free block bitmap 크 기	Ch14
10% reserve (90% 이 상 안 차게)	Seek optimization 트 릭	Ch14
Seek 예시: FCFS 269, SSTF 86, SCAN 169	Disk scheduling 비교	Ch14

OS의 세 가지 역할 ★

역할	의미	예시
Coordinator	자원 공유 조율, 충돌 방 지	CPU scheduling, lock 관리
Illusion Generator	더 좋은 머신처럼 보이게	Virtual memory, multitasking, GUI
Standard Library	공통 함수 모음	File system API, network stack

OS 진화 단계 ★

Phase	시기	특징
Phase I	1950s	Single-program batch, manual loading
Phase II	1960s	Multi-programmed batch, I/O concurrency
Phase III	1970s~	Time-sharing, interactive, modern OS

Interrupt 분류 ★

종류	동기성	발생 원인	예시
----	-----	-------	----

Hardware Interrupt	Asynchronous	외부 신호 (다른 device)	Timer, keyboard, disk I/O 완료
Software Interrupt (Trap)	Synchronous	명령어 실행 결과	syscall, divide-by-zero

Hardware Protection 4가지 ★

보호	메커니즘	무엇을 막는가
Dual Mode	Mode bit + privileged inst	모든 보호의 기반
I/O Protection	I/O instructions = privileged	I/O 독점
Memory Protection	Base/bound register	다른 메모리 침범
CPU Protection	Hardware timer + interrupt	CPU 독점 (무한 루프)

x86 Stack Frame Layout (cdecl) ★

위치	내용
ebp + 4·(N+1)	인자 N
...	...
ebp + 12	인자 2 (y)
ebp + 8	인자 1 (x)
ebp + 4	return address
ebp + 0	saved old ebp ← %ebp
ebp - 4	지역 변수 1
ebp - 8	지역 변수 2
...	... ← %esp (top)

Process State Transition ★

전이	트리거
New → Ready	프로세스 생성 완료
Ready → Running	Dispatched
Running → Ready	Interrupted (timer/preemption)
Running → Waiting	I/O 요청 또는 event 대기
Waiting → Ready	I/O 완료 또는 event 발생
Running → Terminated	exit() 또는 abort()

Process vs Thread ★

항목	Process	Thread
자원 단위	✓ (자원 소유)	✗
실행 단위	✓	✓
주소 공간	독립	공유

스택	자기 것	자기 것
Code/Data/Heap	독립	공유
생성 비용	비쌈	쌘
통신	IPC 필요	공유 메모리 직접

Thread 구현 3가지 ★

방식	Kernel 인 식	Processor 할당 단 위	장점	단점
ULT	모름	Process	Mode switch 없 음, app-specific scheduling	Blocking syscall이 전체 process block
KLT	완전히 앓	Thread	동시 다중 코어 실행, blocking이 thread만 영향	Thread switch마다 2번 mode switch
Combined	부분 (KLT 만)	Thread	ULT의 빠름 + KLT의 multi-core	복잡도

Scheduling Policy 진화 ★

정책	핵심 아이디어	한계
FIFO/FCFS	도착 순	Convoy effect (긴 작업이 짧은 작업 막음)
SJF/SRTF	짧은 burst부터	미래 burst 길이 예측 불가
RR	모두 같은 time slice	Time slice 크기 선택 문제
MLFQ	Priority 큐 + 적응적 time slice	Fair share 불가
GPS	무한히 작은 quantum + weight 비례	구현 불가 (이상)
WRR	Weight $\times$ interval = time slice	Weak fairness
WFQ	Virtual runtime, 가장 작은 것 실행	O(N) overhead
CFS	RB-tree, leftmost = 다음 실행	Linux 표준

Page Replacement Algorithms ★

알고리즘	핵심 아이디어	특성
OPT	미래에 가장 늦게 쓸 page 버림	이론적 최적, 구현 불가
LRU	가장 오래 안 쓴 것 버림	OPT 근사, 정확 구현 비쌘
FIFO	가장 먼저 들어온 것 버림	단순, Belady's anomaly 발생
Random	임의 선택	놀랍게 잘 동작

Clock (2nd chance)	Reference bit 순환 검사	실제 OS 표준
Enhanced Clock	Reference + Dirty bit	dirty page 우선 보존

Memory Management 진화 ★

단계	해결	도입 문제
Base/Bound (1 segment)	단순 보호	Sharing 불가
Multiple Segments	Sharing, 논리 단위	External fragmentation
Paging	External fragmentation 해결	Internal frag, 큰 page table
Paged Segmentation + TLB	두 단계 매핑, table 작게	Translation overhead (해결됨)
Multi-level Page Table	64-bit space 지원	Lookup multi-step

Fragmentation 종류 ★

종류	정의	발생 위치
Internal	할당된 블록 안의 미사용 공간	Block size > payload (alignment 등)
External	전체 free 공간은 충분하나 단일 연속 영역이 없음	malloc, segmentation

Heap Free List 구현 3종 ★

방법	추적 대상	할당 시간	메모리 오버헤드
Implicit list	모든 block	O(전체 블록)	Header만
Explicit list	Free block 만	O(가용 블록)	Header + prev/next
Segregated list	Size class별 list	O(1) 근사	+ 여러 list head

Coalescing 4 Cases (Boundary Tags) ★

Case	좌 (prev)	우 (next)	결과
1	Alloc	Alloc	자기만 free
2	Alloc	Free	자기 + 우
3	Free	Alloc	좌 + 자기
4	Free	Free	좌 + 자기 + 우

Deadlock의 4 Necessary Conditions (Coffman) ★

깨는 prevention 전략

조건	의미	
Mutual Exclusion	자원 공유 불가	자원 공유 가능하게 (대부분 불가)
No Preemption	강제 회수 불가	자원 선점 허용
Hold and Wait	잡은 채 대기	한꺼번에 다 요청
Circular Wait	순환 대기	Ordered request (실용적)

TLB Hit Ratio와 EAT ★

α (hit ratio)	$t_c=10ns, t_m=100ns \rightarrow t_{em}$
0.98	112ns (1.12× of mem)
0.90	120ns (1.2× of mem)
0.50	160ns (1.6× of mem)
0.00	220ns (TLB도 한 번 거침)

Linux Memory Allocation ★

크기	사용 메커니즘	특성
< mmap threshold (보통 64KB or 4MB)	Heap (sbrk 기반)	Free 후 mapping 재사용 (성능 ↑)
≥ mmap threshold	AMO (anonymous mmap)	Free 시 physical frame 즉시 반환 (메모리 효율 ↑)

Major vs Minor Page Fault ★

종류	Page mapping	Disk I/O	비용
Minor	✓	✗	μs 단위
Major	✓	✓	ms 단위 (1000배)

Linux Device 분류 ★

종류	전송 단위	예시	특징
Character	byte	mouse, keyboard, tty	파일처럼 접근, 직접 read/write
Block	block (수 KB)	disk, CD-ROM	파일처럼 접근, buffer cache 사용
Network	packet	eth0	socket API 사용

Device File Major/Minor ★

필드	의미
Type	block 또는 character
Major number	어떤 driver를 쓸지 결정 (cdevsw/bdevsw index)
Minor number	같은 driver 내에서 어느 instance

I/O Routine 4가지 조합 ★

조합	CPU 자원	응답성	사용처
Polling + Blocking	낭비	OK	단순 char device
Polling + Non-blocking	낭비	즉시 return	빠른 retry
Interrupt + Blocking	효율적	sleep	일반 char device
Interrupt + Blocking + Buffer Cache	효율적	두 단계 sleep 가능	block device 표준

File Redirection (Ch13) ★

문법	의미
cmd < file	stdin을 file에서
cmd > file	stdout을 file로 (덮어쓰기)
cmd >> file	stdout을 file에 append
cmd 2> file	stderr를 file로
cmd1 \ cmd2	cmd1 stdout → cmd2 stdin (pipe)

Hard Link vs Symbolic Link (Ch13) ★

	Hard link	Symbolic link
내용	같은 inode	target path string
File system 제약	같은 FS만	다른 FS도 OK
Directory 가능?	✗ (cycle 위험)	✓
Target 삭제 시	데이터 살아있음 (ref count)	dangling link
별도 inode	없음 (공유)	있음
명령	ln file link	ln -s file link

File Access Patterns (Ch14) ★

패턴	정의	예시
Sequential	순서대로 한 block씩	text editor, compiler
Random	임의 순서로 직접	demand paging, BST, DB

Keyed	특정 값 검색	hash table (modern OS 미지원)
--------------	---------	----------------------------

3가지 File Structure (Ch14) ★

구조	핵심	장점	단점
Contiguous	시작 block# + size	seek 최적	external fragmentation
Linked	linked list of blocks	확장 쉬움, no fragmentation	random access 불가
Indexed	pointer array of blocks	seq + random 둘 다 OK	max size 필요
Multi-level (4.3 BSD)	12 direct + 1 indirect + 1 double	작은 파일 빠름, 큰 파일 OK	큰 파일은 extra access

3가지 Block Allocation (Ch14) ★

방식	동작	장단점
Contiguous	생성 시 한 번에	빠르지만 fragmentation
Block-based	사용 시 하나씩	효율적 공간, 많은 seek
Extent-based	(start, size) range 로	seek 최적 + metadata 적음 (modern ext4)

Free Block 추적 (Ch14) ★

방법	동작	장단점
Free list	linked list of free blocks	단순, scalability X (옛 Unix)
Bitmap	1 bit per block	빠른 검색, fixed cost (modern)

File System Cache 3가지 (Ch14) ★

캐시	대상	핵심
Buffer cache	최근 접근 disk block	같은 block 재사용
Page cache	disk에서 온 page	mmap, idle memory 활용
Inode cache	최근 접근 inode	hash table, ref count

Crash Consistency 도구 (Ch14) ★

도구	시기	동작
fsck	부팅 시	전체 disk scan, metadata만 fix, 느림
Journaling	매 op	write-ahead log, transaction 개념

Redo vs Undo Logging (Ch14) ★

	Redo	Undo
핵심		

	committed op 다시 적용	uncommitted op 롤백
로그 내용	변경 사항	원본 데이터
Checkpoint 시점	TxnEnd 후 async	log 후 즉시
사용	ext3, ext4	(이론)

Unix File System Evolution (Ch14) ★

시기	FS	핵심
1974	S5FS	단순, 2~5% bandwidth
1984	FFS	cylinder group, 14~47% bandwidth
1991	LFS	append-only log, 70% write bandwidth
1993	ext2	FFS-like + fsck
1999	ext3	+ journaling
2003	ext4	+ checksum journaling + extent-based

Disk Scheduling Algorithms (Ch14) ★

알고리즘	핵심	특성
FCFS	도착 순	많은 seek
SSTF	nearest first	starvation 위험
SCAN	엘리베이터	balance
C-SCAN	한 방향만 (원형)	boundary fairness
LOOK	C-SCAN 변형	last req까지만

챕터 1-2: 기반

- **Stored-program concept:** 코드와 데이터를 메모리에 함께 저장 (von Neumann 1945)
- **Multiprogramming:** 여러 프로세스가 메모리에 있음 (메모리 관점)
- **Multiprocessing:** 여러 프로세스가 동시 실행 (CPU 관점)
- **FDE cycle:** Fetch → Decode → Execute (instruction의 일생)
- **MMU:** Memory Management Unit, virtual→physical address 변환
- **PSW:** Processor Status Word, mode bit 등 포함
- **Bus master:** bus transaction을 시작할 수 있는 컴포넌트 (CPU, DMA controller)
- **PIC:** Programmable Interrupt Controller, 여러 IRQ를 CPU의 단일 INTR에 다중화
- **System call:** User → Kernel mode 전환을 동반하는 함수 호출

챕터 3: Stack

- **Stack frame (activation record):** 한 함수 호출이 만드는 스택 위 영역
- **Prologue:** pushl %ebp; movl %esp,%ebp; subl \$N,%esp (함수 시작)
- **Epilogue:** movl %ebp,%esp; popl %ebp; ret (함수 종료)
- **Calling convention:** 인자 전달, 반환값, register 저장 책임 등의 규약 (x86은 cdecl)
- **leal:** Load Effective Address — 주소 계산만 하고 메모리 접근 X (덧셈 트릭으로 사용)

챕터 4: Process/Thread

- **PCB**: Process Control Block, 프로세스 정보 저장
- **TCB**: Thread Control Block
- **Context switch**: 한 프로세스에서 다른 프로세스로 CPU 전환
- **Dispatcher**: Mechanism — 실제 전환 수행
- **Scheduler**: Policy — 누구를 다음에 실행시킬지 결정
- **fork()**: 현재 프로세스 복제 (deep copy → COW로 최적화)
- **exec()**: 현재 프로세스 이미지를 새 프로그램으로 교체
- **COW**: Copy-on-Write — fork 시 page 공유, 쓸 때만 복사
- **TLS**: Thread-Local Storage — 각 thread별 독립적 전역 변수

챕터 5: Scheduling

- **CPU burst**: 프로세스가 CPU에서 계속 실행되는 구간 (I/O 사이)
- **Convoy effect**: 긴 process 뒤에 짧은 process가 줄서는 현상 (FIFO)
- **Exponential moving average**: 과거 burst들로 다음 burst 예측 ($\tau = \alpha t + (1-\alpha)\tau_{prev}$)
- **MLFQ**: Multi-Level Feedback Queue — priority 별 큐, time slice 2^k
- **GPS**: Generalized Processor Sharing — fair share의 이상 (불가능)
- **WRR**: Weighted Round Robin — weight 비례 time slice
- **WFQ**: Weighted Fair Queuing — virtual runtime 기반
- **CFS**: Completely Fair Scheduler — Linux의 fair share (RB-tree)
- **Virtual Runtime**: 실제 CPU 시간을 weight로 정규화한 값
- **Nice value**: -20 (높은 priority) ~ 19 (낮은 priority), 0이 default

챕터 6: Synchronization

- **Race condition**: 여러 실행 흐름이 공유 자원 접근 시 타이밍에 따라 결과 달라짐
- **Critical section**: 한 번에 한 process만 실행 가능한 코드 영역
- **Atomicity**: 쪼개지지 않는 연산 (interrupt 불가)
- **Mutual exclusion**: 한 번에 한 process만 어떤 일을 함
- **Semaphore**: 정수 값 + atomic P/V (Dijkstra)
- **P (wait)**: 0 아니면 1 감소
- **V (signal)**: 1 증가
- **Binary semaphore = Mutex**: 0/1만 가짐
- **Counting semaphore**: 임의 음이 아닌 정수
- **Condition variable**: 값 없는 event, wait/signal/broadcast
- **Mesa semantics**: signal 후 조건 재확인 필요 (현대 표준 → while 사용)
- **Hoare semantics**: signal 후 즉시 깨어난 측이 진행 (이론적)
- **Monitor**: ADT + 자동 lock + condition variable

챕터 7: Deadlock

- **Deadlock**: 모든 프로세스가 서로의 자원을 기다려 진행 불가
- **Safe state**: 어떤 순서로든 모든 프로세스의 max를 만족시킬 수 있는 상태
- **Unsafe state**: deadlock 가능성 있는 상태 (반드시 deadlock은 아님)
- **Banker's algorithm**: 자원 할당이 safe state를 유지하는지 검사
- **Resource Allocation Graph**: process-resource 의존 그래프, cycle 검사
- **Ordered request**: 자원에 전역 순서 부여, 그 순서로만 요청 (실용적 prevention)

챕터 8: Linker

- **ELF**: Executable and Linkable Format, .o/.so/a.out의 공통 포맷
- **Section vs Segment**: section은 link time (compile/linking용), segment는 run time (loading용)
- **text/data/bss/rodata**: 코드/초기화된 전역/0초기화 전역/읽기전용 데이터
- **Strong symbol**: function + initialized global
- **Weak symbol**: uninitialized global + extern
- **Symbol Resolution**: 심볼 참조를 정확히 하나의 정의에 연결
- **Relocation**: section 병합 + 상대 주소를 절대 주소로 변환
- **Static library (.a)**: archive of .o, 링크 시 사용된 .o만 들어감
- **Dynamic library (.so)**: load time 또는 runtime에 동적 link, 공유 가능
- **PIC**: Position-Independent Code (-fpic)
- **dlopen/dlsym/dlclose**: runtime dynamic linking API

챕터 9: Dynamic Memory

- **Heap**: 동적 메모리 할당 영역 (sbrk/mmap으로 확장)
- **brk pointer**: heap의 top
- **sbrk(incr)**: brk를 incr만큼 이동
- **Free list**: 사용 중이 아닌 메모리 추적 자료구조
- **Best-fit / First-fit / Next-fit / Worst-fit**: free block 선택 정책
- **Boundary tag**: block 끝에 size/allocated 복사 → bidirectional coalescing 가능
- **Coalescing**: 인접한 free block 합치기 (immediate vs deferred)
- **VMA**: Virtual Memory Area — Linux의 연속 가상 메모리 영역
- **AMO**: Anonymous Memory Object — 익명 mmap으로 만든 VMA
- **GC**: Garbage Collection, mark-and-sweep / reference counter
- **Memory leak**: 사용 안 하는데 회수 안 된 메모리
- **Dangling pointer**: 이미 free된 메모리를 가리키는 포인터

챕터 10: Paging

- **Segment**: 논리적 메모리 단위 (code, data, stack)
- **Page**: 고정 크기 메모리 단위 (보통 4KB)
- **Frame**: physical page (page size와 같음)
- **PTE**: Page Table Entry, PFN + protection bits
- **PTBR**: Page Table Base Register
- **STBR**: Segment Table Base Register
- **TLB**: Translation Lookaside Buffer, address translation cache
- **ASID**: Address Space ID, TLB entry의 process 식별
- **EAT**: Effective Access Time

챕터 11: Demand Paging

- **Virtual memory**: 일부만 메모리에 있어도 실행 가능한 환상
- **Locality**: spatial (인접 주소) + temporal (반복 접근)
- **Knuth's 90/10**: 시간의 90%가 코드의 10%에서
- **Page fault**: 메모리에 없는 page 참조 → trap
- **Major page fault**: disk I/O 동반
- **Minor page fault**: page mapping만 (anonymous page에 대한 첫 접근)
- **Demand paging vs Prepaging**: 참조 시 vs 미리 가져옴
- **Belady's anomaly**: frame 증가에도 fault 증가 (FIFO만)
- **Stack algorithm**: anomaly 면역 (LRU는 stack algorithm)
- **Working set**: 최근 τ 초 동안 참조한 page들

- **Resident set**: 메모리에 있는 process의 page들
- **Balance set**: working set이 memory에 fit하는 active processes 집합
- **Thrashing**: page fault 너무 많아 진전 없음
- **OOM killer**: 메모리 부족 시 low-priority process kill
- **Memory-mapped file (mmap)**: VM 시스템으로 file I/O 처리
- **Hierarchical page table**: multi-level (x86-64 = 4-level)
- **Hashed page table**: VPN → hash → PTE chain
- **Inverted page table**: frame당 1 entry, physical memory에 비례

챕터 12: Device Drivers

- **I/O controller**: 장치 옆 chip, register set + control logic
- **Memory-mapped I/O**: device register를 메모리 주소로 접근
- **Port-based I/O**: 별도 I/O instruction (x86 in/out)
- **DMA**: Direct Memory Access, CPU 없이 큰 데이터 전송
- **Seek time**: head를 track으로 이동
- **Rotational latency**: sector가 head 아래로 오기 대기
- **Transfer time**: sector 읽기/쓰기
- **Cylinder**: 모든 platter의 같은 track 번호 모음
- **cdevsw/bdevsw**: character/block device switch table
- **Strategy routine**: block driver의 block request handler
- **Buffer cache**: block device의 캐싱 layer
- **Top half (ISR)**: short, interrupt disabled, ack + bottom half 활성화
- **Bottom half**: longer, interrupt enabled, deferred execution
- **Polling vs Interrupt-driven**: 반복 검사 vs sleep + wake
- **Blocking vs Non-blocking**: wait vs immediate return

챕터 13: Files and Directories

- **File**: named collection of bytes stored on storage (Unix 정의)
- **“Everything is a file”** (Unix 철학): regular, directory, device, network 모두 file 인터페이스
- **stdin / stdout / stderr**: fd 0/1/2, 모든 process에 기본 제공
- **Redirection**: <, >, >>, 2>, | — shell의 stdin/stdout 조작
- **Directory**: 다른 file과 directory를 담는 special file
- **Dentry (directory entry)**: directory 안의 (name, inumber) 쌍
- **Subdirectory**: directory 안의 directory
- **Pathname**: 절대(/로 시작) vs 상대(working dir 기준)
- **Working directory (cwd)**: 각 process의 현재 디렉토리
- **., .., /**: 현재, 부모, root/구분자
- **Inode (index node)**: file의 metadata (size, time, owner, protection, data block 위치)
- **Inumber (ino)**: inode array의 index
- **Root inumber**: 2 (0, 1은 special purpose)
- **mkfs**: file system 생성 (mkfs -t ext3 /dev/sdb1)
- **Mount**: 새 file system을 기존 tree의 directory에 attach
- **Mount point**: mount되는 directory (새 FS의 root가 됨)
- **Hard link**: 같은 inode를 가리키는 새 directory entry
- **Symbolic (soft) link**: target file의 path string을 담은 별도 file
- **Reference count**: inode 내 link 수 — 0 되면 실제 삭제
- **Dangling link**: target이 사라진 symbolic link

챕터 14: File System

- **Superblock**: file system metadata block
- **Inode blocks**: file metadata 저장

- **Data blocks:** file 실제 데이터
- **VFS (Virtual File System):** Linux의 통합 FS 인터페이스 layer
- **Sequential / Random / Keyed access:** 3가지 access pattern
- **Contiguous file:** 시작 block + size만
- **Linked file:** linked list of blocks
- **Indexed file:** pointer array of blocks
- **Multi-level indexed:** 4.3 BSD — 12 direct + indirect + double indirect
- **Direct pointer:** 작은 파일용 (예: 12개)
- **Indirect block:** pointer들의 block
- **Double indirect:** indirect block 가리키는 block
- **Contiguous allocation / Block-based / Extent-based:** 3가지 block 할당
- **Extent:** 연속 block range, (start, size) 쌍
- **Free list / Bitmap:** 2가지 free block 추적
- **Buffer cache:** disk block 캐시
- **Page cache:** disk page 캐시 (mmap과 통합)
- **Inode cache:** inode 캐시 (hash table)
- **Seek optimization:** 인접 block 배치 + 10% reserve
- **Crash consistency:** power loss/crash 시 atomic update
- **Data integrity:** disk corruption에도 데이터 같게
- **fsck:** file system checker, 부팅 시 metadata 복구
- **lost+found:** 고아 inode 보관 directory
- **Journaling:** write-ahead log로 crash 복구
- **Write-ahead logging:** DB에서 빌려옴, log 먼저 disk에
- **Transaction (FS):** file system 변경의 commit-or-abort 단위
- **TxnBegin / TxnEnd:** transaction 경계 record
- **Commit point:** no-turning-back 시점
- **Checkpointing:** log의 변경을 in-place로 반영
- **Redo logging:** committed op 다시 적용 (ext3/4 표준)
- **Undo logging:** uncommitted op 롤백
- **Checksum journaling:** log 내용에 checksum (ext4)
- **S5FS (1974):** Ken Thomson, 옛 Unix FS
- **FFS (1984):** Berkeley Fast FS, cylinder group, disk awareness
- **LFS (1991):** Log-structured, append-only, 70% bandwidth
- **Ext2/3/4:** Linux 표준 진화 (fsck → journaling → checksum)
- **Cylinder group:** FFS의 관련 파일 grouping 단위
- **Fragment:** FFS의 sub-block (작은 파일용)
- **Rotational positioning (rotdelay):** 다음 read 시 head가 도착하도록 interleave
- **Disk scheduling:** I/O request queue 정렬
- **FCFS, SSTF, SCAN, C-SCAN, LOOK:** scheduling 알고리즘
- **SPTF (Shortest Positioning Time First):** rotational 고려한 변형

보충: Transaction & ACID

- **Transaction:** 하나의 논리적 작업 단위 (전부 성공 or 전부 실패)
- **Atomicity (A):** 부분 실행 안 됨
- **Consistency (C):** invariant 유지
- **Isolation (I):** 동시 실행 transaction 간 간섭 없음 (race condition 방지)
- **Durability (D):** commit 후 결과 영구적
- **Serializability:** 동시 실행 결과가 어떤 순차 실행과 같음

토픽 1: OS의 진화 3단계와 동기 (Ch1)

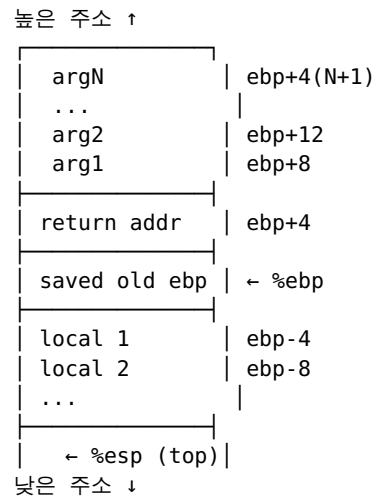
3. CPU Protection: - Hardware timer + interrupt - 일정 시간마다 timer interrupt → kernel이 다음 process 선택 - 무한 루프 process도 강제 회수 가능

이 네 가지가 상호 의존적 — dual mode가 없으면 나머지 세 개도 무의미.

토픽 5: x86 cdecl Calling Convention (Ch3)

인자 전달: 모두 스택에 (오른쪽에서 왼쪽 순서로 push) 반환값: **%eax Caller-saved**: %eax, %ecx, %edx **Callee-saved**: %ebx, %esi, %edi, %ebp

Stack Frame 구조:



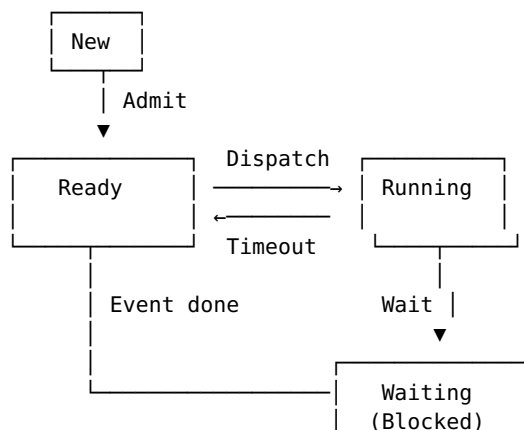
Prologue (함수 시작):

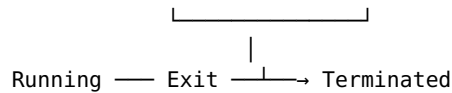
```
pushl %ebp ; 호출자의 ebp 저장
movl %esp, %ebp ; 새 frame의 시작점
subl $N, %esp ; 지역 변수 공간 확보
```

Epilogue (함수 종료):

```
movl %ebp, %esp ; esp를 frame 시작으로 복원
popl %ebp ; 호출자의 ebp 복원
ret ; return address pop, PC = ret addr
```

토픽 6: Process State Transition (Ch4)





전이의 트리거를 외우는 것이 핵심.

토픽 7: Context Switch와 fork/exec/wait (Ch4)

Context Switch 단계: 1. 현재 process의 register들을 PCB에 저장 2. Scheduler가 다음 process 선택 3. 선택된 process의 PCB에서 register 복원 4. PC를 새 process의 다음 instruction으로

fork(): - 현재 process의 완전한 복사본 생성 - 부모 return: child PID, 자식 return: 0 - 실제 구현: **COW(Copy-on-Write)** - page table만 복사, page는 공유, write 시 분리

exec(): - 현재 process의 메모리 이미지를 새 **program**으로 교체 - PID 유지, 같은 PCB - 보통 fork() 직후 자식이 exec() 호출

wait(): - 부모가 자식 종료 대기 - 안 하면 **zombie process** 발생 (PCB는 남아있음)

토픽 8: Thread 구현 3가지 (Ch4)

ULT (User-Level Threads): - Library가 user space에서 관리 - Kernel은 process만 봄 - 장점: mode switch 없음, custom scheduling - 단점: 한 thread의 blocking syscall이 전체 process block

KLT (Kernel-Level Threads): - Kernel이 thread를 직접 관리 - 장점: 한 thread blocking이 다른 thread 영향 X, multi-core 활용 - 단점: thread switch마다 2번 mode switch

Combined (M:N): - M user threads → N kernel threads로 매핑 - 양쪽의 장점 결합, 복잡도 ↑

토픽 9: Scheduling Algorithms (Ch5)

SJF의 burst 예측 — Exponential Moving Average:

$$\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$$

- t_n : 실제 n번째 burst 길이
- τ_n : 예측치
- α : smoothing factor (0.5 ~ 0.8 보통)

예시 문제: $\tau_0 = 10$, $\alpha = 0.5$, burst = 6, 4, 6, 4, 13... - $\tau_1 = 0.5 \times 6 + 0.5 \times 10 = 8$ - $\tau_2 = 0.5 \times 4 + 0.5 \times 8 = 6$ - $\tau_3 = 0.5 \times 6 + 0.5 \times 6 = 6$ - $\tau_4 = 0.5 \times 4 + 0.5 \times 6 = 5$ - $\tau_5 = 0.5 \times 13 + 0.5 \times 5 = 9$

MLFQ 규칙: 1. 새 process는 최고 **priority**, 짧은 time slice ($2^0=1$) 2. Time slice 다 쓰고 block 안 되면 → priority 1 강등, time slice 2배 3. I/O로 block되면 priority 유지 (= I/O 친화적)

이 규칙이 자연스럽게 **I/O-intensive**는 **priority** 유지, **CPU-intensive**는 강등시킴.

CFS의 동작: 1. 각 task에 **virtual runtime (VR)** 부여 2. **VR**이 가장 작은 **task** = 가장 뒤쳐진 **task** = 다음 실행 3. RB-tree에 정렬 보관, **leftmost node** = **O(1)** 조회 4. Weight가 클수록 VR이 천천히 증가 (실제 시간을 weight로 나눔)

Nice 10% rule: nice 1단계 = CPU 사용량 $\pm 10\%$

토픽 10: Producer/Consumer Pattern (Ch6)

가장 중요한 동기화 패턴.

```
Semaphore buf_avail = N;    // 빈 슬롯 수
Semaphore data_avail = 0;   // 데이터 있는 슬롯 수

Producer() {
    P(buf_avail);           // 빈 슬롯 기다림
    buf = data;             // 데이터 쓰기
    V(data_avail);          // 데이터 슬롯 증가
}

Consumer() {
    P(data_avail);          // 데이터 기다림
    data = buf;             // 데이터 읽기
    V(buf_avail);           // 빈 슬롯 증가
}
```

핵심 패턴: - 자원 생성 시 V() (signal) - 자원 소비 시 P() (wait) - Producer와 Consumer가 반대 방향으로 P/V 짝맞춤

토픽 11: Condition Variable 사용 패턴 (Ch6)

```
pthread_mutex_lock(&mutex);
while (!condition_met)
    pthread_cond_wait(&cond, &mutex);
// 여기서 condition_met이 보장됨
... critical section work ...
pthread_mutex_unlock(&mutex);
```

중요한 점들: - pthread_cond_wait()는 **mutex**를 인자로 받음 (unlock + wait 가 atomic해야 함, 안 그러면 missed wakeup 발생) - 반드시 **while** (not if)
— Mesa semantics: signal 후 조건 다시 확인 필요 (spurious wakeup 가능)
- Wait가 호출되면: ① mutex unlock ② sleep ③ signal 받으면 깨어남 ④ mutex 재획득

토픽 12: Banker's Algorithm (Ch7)

자료구조: - Available[m]: 가용 자원 수 - Max[n][m]: 최대 필요량 - Allocation[n][m]: 현재 할당 - Need[n][m] = Max - Allocation

Safety Algorithm:

```
Work = Available
Finish[i] = false for all i
```

REPEAT:

```
Find i: Finish[i]==false AND Need[i] ≤ Work
if found:
    Work += Allocation[i]
    Finish[i] = true
```

```

else:
    break

if all Finish[i] == true: SAFE
else: UNSAFE

```

Resource Request Algorithm: 1. Request[i] ≤ Need[i] 검사 (Max 초과 X) 2. Request[i] ≤ Available 검사 (당장 가능?) 3. 가상 할당 후 Safety Algorithm 실행 4. Safe면 진짜 할당, Unsafe면 대기

예시 문제 (5 processes, 3 resource types A/B/C):

	Max	Allocation	Need = Max-Alloc	Available
P0	7 5 3	0 1 0	7 4 3	3 3 2
P1	3 2 2	2 0 0	1 2 2	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

Safe sequence 찾기: Work=(3,3,2)에서 시작 - P1 Need(1,2,2) ≤ (3,3,2) ✓
→ Work=(5,3,2) - P3 Need(0,1,1) ≤ (5,3,2) ✓ → Work=(7,4,3) - P4
Need(4,3,1) ≤ (7,4,3) ✓ → Work=(7,4,5) - P0 Need(7,4,3) ≤ (7,4,5) ✓ →
Work=(7,5,5) - P2 Need(6,0,0) ≤ (7,5,5) ✓ → Work=(10,5,7)

Safe sequence: {P1, P3, P4, P0, P2} ✓

토픽 13: Linker의 두 작업 (Ch8)

작업 1: Symbol Resolution - 각 심볼 참조를 정확히 하나의 정의에 연결 -
Strong/Weak 규칙: - Strong 중복 → linker error - Strong + Weak →
Strong 선택 - Weak 중복 → 임의 선택

작업 2: Relocation - 여러 .o의 같은 종류 section을 단일 **section**으로 병합
- 상대 주소를 실제 메모리 주소로 변환 - Relocation table을 사용해 어디를
patch할지 결정

ELF Object File 구조:

ELF header
Program header table (executable에 필수)
.text ← code
.rodata ← read-only data
.data ← initialized globals
.bss ← uninitialized globals (header only!)
.symtab ← symbol table
.rel.text ← .text relocation info
.rel.data ← .data relocation info
.debug
Section header table (relocatable에 필수)

토픽 14: Static vs Dynamic Linking (Ch8)

Static: - Library의 사용된 .o가 실행 파일에 복사됨 - 100개 프로세스 = 100
개 libc 복사본 - 공간/메모리 낭비 - Library 업데이트 시 모든 응용 재링크

Dynamic: - .so 파일을 load time 또는 runtime에 link - 모든 프로세스가 한
copy 공유 - Library 업데이트가 즉시 반영 - **PIC** (-fpic) 필요: 어디든 로드 가
능한 코드

Command line 순서 함정:

```
# 잘못된 순서 (library가 앞)
gcc -static -o prog -L. -lvector main.o # X undefined ref

# 올바른 순서 (library가 뒤)
gcc -static -o prog main.o -L. -lvector # ✓
```

Linker는 순차 스캔하면서 unresolved를 추적하므로 **library**는 항상 뒤에.

토픽 15: Heap의 5가지 설계 질문 (Ch9)

좋은 heap allocator를 만들려면:

1. **Free**할 때 크기를 어떻게 아는가? → **Header**에 size 저장
2. **Free block**을 어떻게 추적? → Implicit/Explicit/Segregated list
3. 여러 후보 중 어떤 **block** 선택? → First-fit/Best-fit/Next-fit
4. 남는 공간 처리? → **Split** (남은 부분을 새 free block으로)
5. **Free** 시 어떻게 **insert**? → LIFO 또는 address-ordered

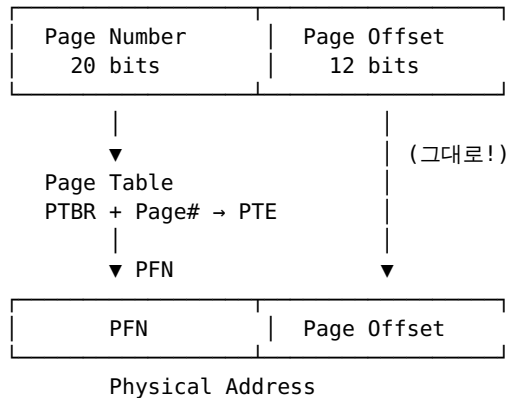
토픽 16: Coalescing 4 Cases (Ch9)

Case 1: m1(Alloc) | n | m2(Alloc) → m1 | n(Free) | m2 (단독)
 Case 2: m1(Alloc) | n | m2(Free) → m1 | n+m2(Free) (우만 합침)
 Case 3: m1(Free) | n | m2(Alloc) → m1+n(Free) | m2 (좌만 합침)
 Case 4: m1(Free) | n | m2(Free) → m1+n+m2(Free) (둘 다)

Boundary Tag가 필수 — block 끝에 size 복사가 있어야 좌측 block을 찾을 수 있음.

토픽 17: Paging Address Translation (Ch10)

Logical Address (32-bit, 4KB page):



Page offset은 변환 안 됨 — page 안의 위치는 logical/physical 같음.

토픽 18: TLB와 EAT (Ch10)

TLB hit ratio가 α 일 때:

$$t_{em} = \alpha \cdot (t_c + t_m) + (1 - \alpha) \cdot (2t_c + 2t_m)$$

- Hit: TLB 검색 + 메모리 1번
- Miss: TLB 검색 + page table 1번 + 메모리 1번 + TLB 채우기

예시 계산 ($t_c = 10\text{ns}$, $t_m = 100\text{ns}$, $\alpha = 0.98$): - Hit case: $10 + 100 = 110\text{ns}$ - Miss case: $20 + 200 = 220\text{ns}$ - Average: $0.98 \times 110 + 0.02 \times 220 = 107.8 + 4.4 = 112.2\text{ ns}$

TLB 없으면 매번 200ns → 약 1.78배 빨라짐

Context switch 처리: 1. **Flush TLB** (전체 무효화) — 단순하지만 비쌈 2. **ASID 사용** — TLB entry에 process ID도 저장. 다른 process는 자동 미스

토픽 19: Demand Paging Page Fault Flow (Ch11)

1. Process가 page에 접근
2. MMU가 PTE 확인 - valid bit = 0
3. Trap → kernel
4. Kernel이 page table entry 확인
 - 진짜 page fault인가? (예: invalid 주소면 SIGSEGV)
5. 빈 frame 할당 (없으면 victim 선택 + page replacement)
 - Dirty page면 disk에 write-back
6. 디스크에서 page 읽기 (← 가장 느림, ms 단위)
7. PTE 업데이트: PFN 채우기, valid bit = 1
8. TLB에 entry 추가
9. 중단된 instruction 재시작

각 단계가 비용이지만 가장 큰 비용은 **disk I/O** (5번~6번).

토픽 20: Page Replacement Algorithms (Ch11)

Reference string: A B C A B D A D B C B, **3 frames**

Step	Ref	FIFO	OPT	LRU
1	A	[A,,] M	[A,,] M	[A,,] M
2	B	[A,B,] M	[A,B,] M	[A,B,] M
3	C	[A,B,C] M	[A,B,C] M	[A,B,C] M
4	A	[A,B,C] H	[A,B,C] H	[A,B,C] H
5	B	[A,B,C] H	[A,B,C] H	[A,B,C] H
6	D	[D,B,C]→A out	[A,B,D]→C out	[A,B,D]→C out
7	A	[D,A,C]→B out	[A,B,D] H	[A,B,D] H
8	D	[D,A,C] H	[A,B,D] H	[A,B,D] H
9	B	[D,A,B]→C out	[A,B,D] H	[A,B,D] H
10	C	[C,A,B]→D out	[C,B,D]→A out	[C,B,D]→A out
11	B	[C,A,B] H	[C,B,D] H	[C,B,D] H

Faults: FIFO=8, OPT=5, LRU=5

Belady's Anomaly (FIFO만): frame을 늘려도 fault가 증가할 수 있음.

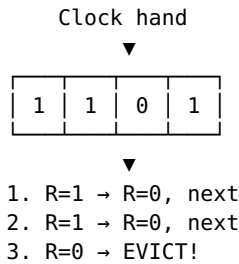
토픽 21: Clock Algorithm (Ch11)

LRU의 효율적 근사. **Reference bit(R)** 활용:

On page eviction:

1. Clock hand position에서 시작
2. $R == 0?$ → 이 page를 evict
3. $R == 1?$ → $R = 0$ 으로 set, hand 다음으로
4. Back to step 2

시각화:



장점: $O(1)$ 평균, 매우 효율적 단점: 모든 R=1이면 FIFO처럼 동작

토픽 22: Working Set Model (Ch11)

정의: 최근 τ 초 동안 참조한 page들

Working set strategy: - 프로세스는 자기 **working set**이 메모리에 다 있어야 실행 가능 - 안 들어가면 swap out, 다른 process가 실행

Balance set: - Working set이 메모리에 fit하는 process 집합 = "active" - 나머지는 swap out

→ **Thrashing** 방지 + multiprogramming degree 최대 유지

토픽 23: 두 종류 Page Fault (Ch11)

Minor page fault: - Anonymous page의 첫 접근 - Page mapping만 (disk I/O 없음) - 비용: μs 단위 - 예: malloc() 후 처음 쓸 때

Major page fault: - File-backed page거나 swap out된 page - **Disk I/O** 동반 - 비용: ms 단위 (1000배) - 예: 디스크에서 처음 실행 파일 로드

토픽 24: Disk vs Flash 특성 (Ch12)

Disk (HDD): - 기계적 부품 (head, platter) - **Seek time:** 0~50ms (head 이동) - **Rotational delay:** 0~16ms - **Transfer:** 8~40 μs - 핵심: **seek/rotation이 transfer의 1000배 느림** → disk scheduling 중요

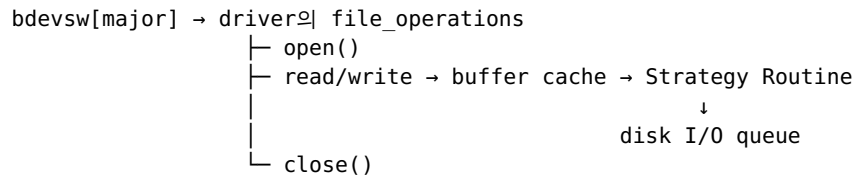
Flash (SSD): - 전자식, 기계 부품 없음 - **Read:** 20 μs (빠름) - **Write:** 200 μs (10배 느림) - **Erase:** 2000 μs (또 10배 느림) - 제약: - **Erase-before-write:** page 재기록 불가, block 전체 erase 후만 - **Wear-out:** erase 횟수 한계 → wear leveling - 해결: **Log-structured** (변경분을 다른 위치에 기록, 원본 invalid 표시)

토픽 25: Device Driver 구조 (Ch12)

Character Device Driver:

```
cdevsw[major] → driver의 file_operations
├─ open()
├─ read()
├─ write()
├─ ioctl()
└─ close()
```

Block Device Driver:



Block device에는 **buffer cache** 추가 — 같은 block 반복 접근 시 hit, batching/scheduling 가능.

토픽 26: Two-Level Interrupt Handling (Ch12)

Top half (ISR): - 짧음, interrupt disabled 상태에서 실행 - 주요 동작: device ack, bottom half 활성화 - 나머지 작업은 bottom half로 미룸

Bottom half: - 길어도 됨, interrupt enabled 상태에서 실행 - Deferred execution (조금 나중에 실행 가능) - 실제 데이터 전송, queue 처리 등

왜 분리? 인터럽트가 빠르게 연속 발생할 때 ISR이 길면 시스템 응답성 저하. 짧은 ISR + 긴 bottom half로 분리하면 다른 인터럽트도 잘 처리.

토픽 27: Path Name Parsing (Ch13)

/a/b/c 같은 **path**가 어떻게 **disk**의 데이터가 되는가

```

Inode 2 (root):  Contains <"a", 5>
    ↓
Inode 5 (dir):   Contains <"b", 7>
    ↓
Inode 7 (dir):   Contains <"c", 14>
    ↓
Inode 14 (file): 실제 데이터

```

핵심: - **Root inumber = 2** (0, 1은 special) - Directory = (name, inumber) 쌍들을 담은 **special file** - Path parsing = 재귀적 **inode lookup**
 - 깊은 path = 더 많은 disk read → dentry cache로 최적화

토픽 28: Hard Link vs Symbolic Link (Ch13)

Hard link — 같은 inode 공유:

```

$ ln file file_link
$ ls -li file file_link
671158084 file
671158084 file_link    # 같은 inode!

```

삭제 동작: - 각 inode에 **reference count** - rm은 directory entry만 제거 + count 감소 - **count == 0** 되어야 실제 데이터 해제

Symbolic link — target path를 담은 별도 file:

```

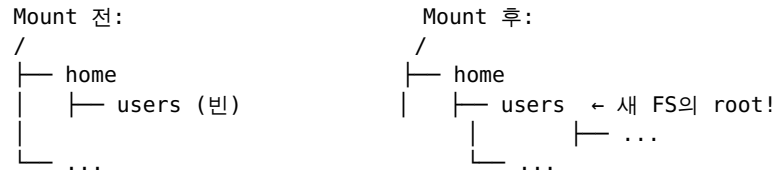
$ ln -s file file_link
$ stat file_link
symbolic link

```

차이점: - Hard: same FS only, no dir, no dangling - Symbolic: cross-FS, dir OK, **dangling** 가능

토픽 29: Mount의 동작 (Ch13)

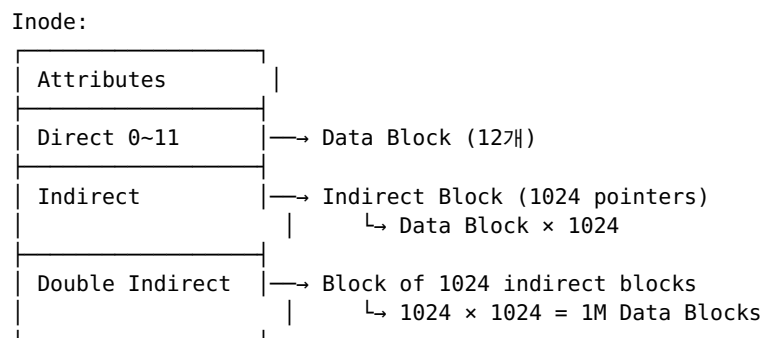
```
$ mkfs -t ext3 /dev/sdb1
$ mount -t ext3 /dev/sdb1 /home/users
```



핵심 통찰: 컴퓨터의 여러 file system이 사용자에게는 하나의 통합 **tree**처럼 보임.

토픽 30: 4.3 BSD Multi-level Indexed File (Ch14) ★

4 KB block, 4 B index 가정:

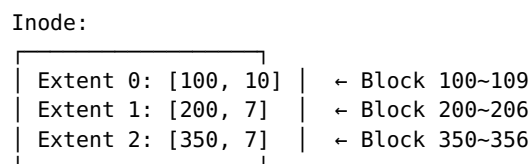


파일 크기 한계: - Direct: $12 \times 4\text{KB} = 48\text{ KB}$ - + Indirect: $1024 \times 4\text{KB} = 4\text{ MB}$ - + Double indirect: $1\text{M} \times 4\text{KB} = 4\text{ GB}$ - + Triple indirect (있다면): $1\text{G} \times 4\text{KB} = 4\text{ TB}$

Trade-off: - 작은 파일 ($< 48\text{KB}$): inode 한 번 → 데이터 바로 (빠름) - 큰 파일: 2~3번 추가 inode access

토픽 31: Extent-based Allocation (Ch14)

Extent = 연속 block range, (**start, size**) 쌍으로 압축.



장점: - Sequential 효율 (연속 block) - **Metadata** 작음 (큰 파일도 몇 개의 extent만) - 큰 physical write 가능

사용: Linux ext4, VxFS, QFS.

토픽 32: Journaling — Write-Ahead Logging (Ch14) ★

Transaction의 **commit-or-abort** 의미를 file system에:

1. TxnBegin write (async, 고유 ID)
2. File system 변경 사항 write
 - ↳ TxnBegin + 변경사항이 disk에 있어야 다음

3. TxnEnd write
 - ↳ disk에 TxnEnd 완료 = transaction committed
4. Checkpoint: 실제 file system 변경 적용 (async)

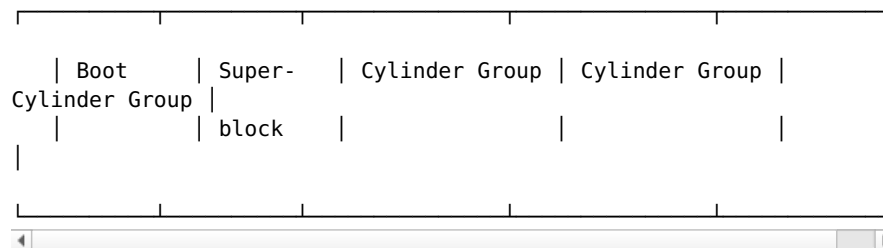
Crash Recovery: 1. Log scan 2. TxnBegin → TxnEnd 매칭 3. 둘 다 있으면 (committed) → 변경 적용 4. TxnEnd 없으면 (uncommitted) → 무시

ext3 vs ext4: - ext3: synchronous log write → 느림 - **ext4: checksum journaling** → log 전체 한 번에 write, recovery 시 checksum 검증

토픽 33: FFS의 Cylinder Group (Ch14) ★

FFS의 Key idea: "Keep related stuff together!"

FFS Partition:



각 cylinder group:

S/B	Inode Bitmap	Data Bitmap	Inodes	Data Blocks
-----	--------------	-------------	--------	-------------

할당 규칙: - 한 directory의 파일 inode들을 같은 **group**에 - 새 directory는 부모와 다른 **group**에 (분산) - 파일 data block은 **inode**와 같은 **group**에

큰 파일 처리: - 처음 12 block: 한 group - **48 KB**부터 **group** 변경 (direct pointer 다 채워서) - **매 1 MB**마다 또 group 변경 (한 group을 독점하지 않게)

효과: S5FS의 2% bandwidth → **FFS의 14~47%**.

토픽 34: Disk Scheduling 비교 (Ch14) ★

예시: Cylinder 0, 53, 14, 27, 2, 31, 85, 30 요청. Head 초기 = 1.

알고리즘	처리 순서	Total seek
FCFS	0, 53, 14, 27, 2, 31, 85, 30	269
SSTF	0, 2, 14, 27, 30, 31, 53, 85	86
SCAN	2, 14, 27, 30, 31, 53, 85, 0	169
C-SCAN	2, 14, 27, 30, 31, 53, 85, 0	169

Trade-off: - **SSTF:** 가장 빠르지만 **starvation** 위험 (멀리 있는 request) - **SCAN (elevator):** 한 방향씩 왕복, balance - **C-SCAN:** 한 방향만, 끝에서 시작점으로 jump → boundary fairness

Modern: - **LOOK:** C-SCAN 변형, last request까지만 이동 - **SPTF/SATF:** rotational latency도 고려 - **Flash:** mechanical seek 없음 → 다른 전략 (wear leveling 등)

🔑 챕터 간 연결 정리

- **Ch1 (base/bound) → Ch10 (multiple segments → paging):** 메모리 관리의 점진적 진화
 - **Ch2 (interrupt) → Ch11 (page fault):** page fault는 trap의 한 종류
 - **Ch2 (hardware protection) → Ch10 (memory protection):** PTE의 protection bits
 - **Ch3 (stack) → Ch4 (process):** 각 process는 자기 stack
 - **Ch4 (process) → Ch6 (sync):** thread 공유 메모리에서 race
 - **Ch6 (sync) → Ch7 (deadlock):** lock 여러 개 잡으면 deadlock
 - **Ch8 (linker) → Ch9 (heap):** data/bss는 정적, heap은 동적
 - **Ch9 (heap) → Ch11 (demand paging):** AMO가 anonymous mmap
 - **Ch10 (paging) → Ch11 (demand paging):** paging의 valid bit 활용
 - **Ch11 (demand paging) → Ch12 (device driver):** page fault 시 disk driver 호출
 - **Ch9 (heap) → Ch13 (link):** reference counter는 hard link와 같은 원리
 - **Ch12 (device driver) → Ch13 (file system):** device file(/dev/...) 이 driver와 FS의 다리
 - **Ch13 (files/dirs) → Ch14 (file system):** 추상화의 구현. inode가 실제로 어떻게 disk에
 - **Ch9 (heap) → Ch14 (free block):** 둘 다 free list/bitmap으로 추적 (같은 패턴)
 - **Ch11 (page cache) → Ch14 (page cache):** mmap이 FS와 VM 통합
 - **Ch12 (buffer cache) → Ch14 (buffer cache):** block device driver의 buffer cache 동일
 - **ACID 보충 → Ch14 (journaling):** transaction의 ACID를 file system에 적용
-